

Supplementary Material for MV-IVF: Multi-Vector Retrieval via Multi-Vector Clustering

A PROOF OF PROPOSITION 1

PROOF OF PROPOSITION 1. By definition of Chamfer distance, for any center-set C ,

$$\sum_{P \in \Pi_j} \text{Ch}(P, C) = \sum_{P \in \Pi_j} \frac{1}{|P|} \sum_{x \in P} \min_{c \in C} d(x, c).$$

Reindexing the double sum over the multiset $\text{Flatten}(\Pi_j)$, the point $x \in P$ receives weight $1/|P|$. Hence,

$$\sum_{P \in \Pi_j} \text{Ch}(P, C) = \sum_{x \in \text{Flatten}(\Pi_j)} w(x) \min_{c \in C} d(x, c),$$

where $w(x) = 1/|P|$ for x coming from P . Therefore, minimizing the Chamfer $(1, s)$ -median objective over all center-sets C with $|C| \leq s$ is exactly the weighted s -median problem on the flattened multiset $\text{Flatten}(\Pi_j)$. $\square$

B EXPERIMENTS (CONTINUED)

In this section, we continue the discussion of the experiments from the main paper.

B.1 Baselines and Parameter Settings

We compare MV-IVF against three methods: SVH, MUVERA, and MV-Vamana. SVH is a single-vector candidate-generation baseline: it indexes the flattened document embeddings with a single-vector ANN index, retrieves candidate documents through their individual embeddings, and reranks the resulting candidates using multi-vector scoring. MUVERA reduces each point cloud to a fixed-dimensional encoding and performs candidate generation using a single-vector ANN index over these encodings. MV-Vamana is our native graph-based baseline, obtained by building a Vamana-style graph directly over document point clouds.

All four methods are implemented in MVSIC and use the same document/query embeddings, scoring kernels, and evaluation pipeline. For SVH and MUVERA, the underlying single-vector ANN index is Vamana. For MUVERA, we use the parameter settings recommended in [3]; for all Vamana graphs, we use standard settings from ParlayANN [5]. For each method, we sweep the search-time parameters listed in Table 1 and report the Pareto frontier over all resulting operating points. Thus, the main comparisons are based on the best recall–efficiency tradeoff achieved by each method over its specified parameter grid, rather than on a single hand-tuned setting.

B.2 Quantization

We evaluate several quantization paradigms used throughout the paper. In addition to the TQ, Rand-1bit, and Rand-8bit variants described in Section 5, we also compare against PQ FastScan-style kernels [1, 2] and RaBitQ [4]. For RaBitQ, we use the publicly available RaBitQ library implementation [4].

Single-vector over-retrieval benchmark. We first evaluate the quantizers on single-vector inner-product search. For each quantized scorer, we retrieve the top k' candidates under the approximate scores and report Recall-10@ k' with respect to the exact top-10 neighbors. This measures how much over-retrieval is needed for the quantized scores to preserve the true nearest neighbors. Figure 1 reports the resulting recall curves.

In the plot labels, FS-4b, FS-8b, etc. denote the block-size variants used by our FastScan implementation; in this notation, FS-4b corresponds to one bit per coordinate, while FS-8b corresponds to one half-bit per coordinate. We also include both Rand-1bit and Rand-1bit-asym: Rand-1bit quantizes both database and query vectors, while Rand-1bit-asym uses an asymmetric query variant. We observe that Rand-8bit and RQ-8bit preserve quality extremely well, making them suitable as high-fidelity reranking quantizers. RQ-4bit and TQ-4bit provide a good medium-quality tradeoff and are well suited for candidate generation, while Rand-1bit, RQ-1bit, FS-2b, and FS-4b are lower-quality but substantially cheaper scorers.

Multi-vector over-retrieval benchmark. We next evaluate the same quantizers inside the multi-vector retrieval pipeline. Figure 2 reports the resulting recall–over-retrieval curves.

An interesting observation is that the quantizers perform comparatively better in the multi-vector setting than in the single-vector setting. One reason is that MaxSim aggregates many query-token contributions into a single query-document score. As a result, errors in individual inner-product estimates can partially smooth out at the level of the full point-cloud score. This makes even relatively low-precision quantizers useful for coarse multi-vector scoring, especially when the goal is to recover a sufficiently large candidate set rather than exactly match the full-precision ordering.

B.3 Query Compression

Multi-vector scoring is linear in the number of query embeddings, and many queries contain redundant token vectors. We therefore compress each query point cloud before scoring. Given a query Q , both compression methods considered below produce a partition $\mathcal{B} = \{B_1, \dots, B_r\}$ of the query embeddings, and replace each block B_i by a single representative $\bar{q}_i := \sum_{q \in B_i} q$. The compressed query is then $\bar{Q} = \{\bar{q}_1, \dots, \bar{q}_r\}$. Using sums rather than averages preserves the scale of the original MaxSim score: a block with more query embeddings contributes proportionally more to the final score. Larger compression thresholds produce fewer representatives and faster scoring, but can reduce retrieval quality.

Ball Carving. Ball carving is the greedy query-compression method used by MUVERA [3]. Given a distance threshold τ , the algorithm repeatedly selects an unassigned query embedding q , forms a block consisting of all currently unassigned embeddings within distance τ of q , replaces this block by the sum of its vectors, and removes the block from the active set. This process continues until all query embeddings have been assigned. Thus, the number of compressed

Method	Build parameters	Search parameters
MV-IVF	- Recursive MV-Lloyd tree with automatic branching. - If a node contains n_u point clouds and $b = 0$, then $b_u = \min\{\lceil \sqrt{n_u} \rceil, \lceil cn_u / \ell_{\max} \rceil\}$. 5 Lloyd iterations; at most 100 point clouds per update cluster. - Build-time assignments use TQ-8bit scoring. $\ell_{\max} = 500$.	$nprobe \in \{1, 2, 4, \dots\}$. - TQ-4bit for greedy search; Rand-1bit for leaf scoring; Rand-8bit for reranking. - Query compression: Ward’s method with $\tau = 0.5$. $num_rerank \in \{20, 100, 500\}$ for $K = 10$. $num_rerank \in \{200, 1000, 5000\}$ for $K = 100$.
MUVERA	$d_{FDE} = 10240$, except MS MARCO where $d_{FDE} = 2560$. - Vamana graph with $R = 200$, $L_{\text{build}} = 600$, and $\alpha = 1.0$.	- Search beam $L \in \{16, 32, \dots\}$. - TQ-4bit for graph search; Rand-8bit for reranking. $num_rerank \in \{500, 1000\}$ for $K = 10$. $num_rerank \in \{5000, 10000\}$ for $K = 100$.
SVH	- Vamana graph over flattened individual embeddings. $R = 64$, $L_{\text{build}} = 128$, and $\alpha = 1.0$.	- Search beam $L \in \{16, 32, \dots\}$. - TQ-4bit for graph search; Rand-8bit for reranking. $num_rerank \in \{50, 100, 500\}$ for $K = 10$. $num_rerank \in \{1000, 5000\}$ for $K = 100$.
MV-Vamana	- Native Vamana graph over document point clouds. $R = 64$, $L_{\text{build}} = 128$, and $\alpha = 1.0$.	- Search beam $L \in \{16, 32, \dots\}$. - TQ-4bit for graph search; Rand-8bit for reranking. $num_rerank \in \{20, 50\}$ for $K = 10$. $num_rerank \in \{200, 500\}$ for $K = 100$.

Table 1: Build parameters and search-time grids used to generate the Pareto curves in Sections 6. Early cut-off was applied when latency went above 100ms, or QPS dropped below 10.

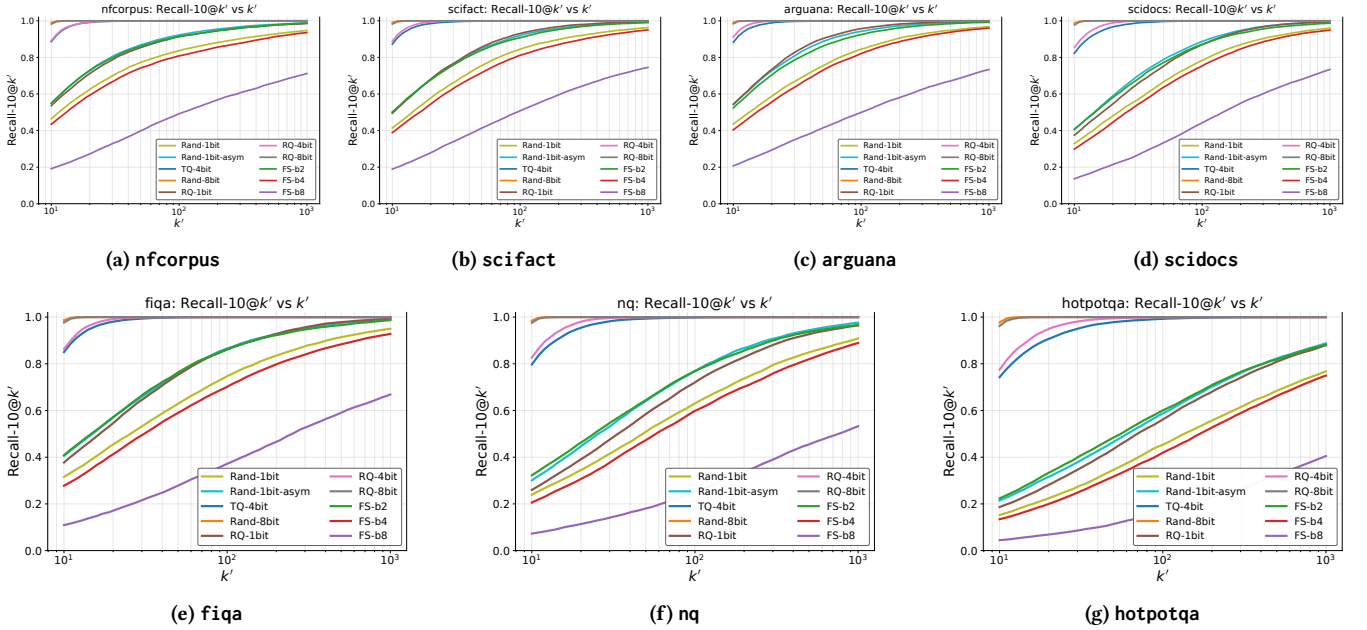


Figure 1: Single-vector over-retrieval quality across datasets. Each panel shows recall as a function of the candidate budget k' .

query vectors is not fixed in advance; it is determined by the geometry of the query and the threshold τ .

Ward’s Method. We also consider compression based on Ward’s agglomerative clustering [6]. Each query embedding is initialized as a singleton cluster. We then repeatedly merge the pair of clusters that causes the smallest increase in within-cluster variance, stopping once the next merge exceeds a threshold τ . As with ball

carving, each final cluster is represented by the sum of its embeddings. Ward compression tends to form more balanced and coherent groups of query vectors, and in our experiments gives a slightly better quality–compression tradeoff.

Discussion. Figure 3 compares the two compression methods as the threshold is varied. Both methods are effective: moderate compression reduces the number of query embeddings substantially with little effect on recall. We use Ward’s method with threshold

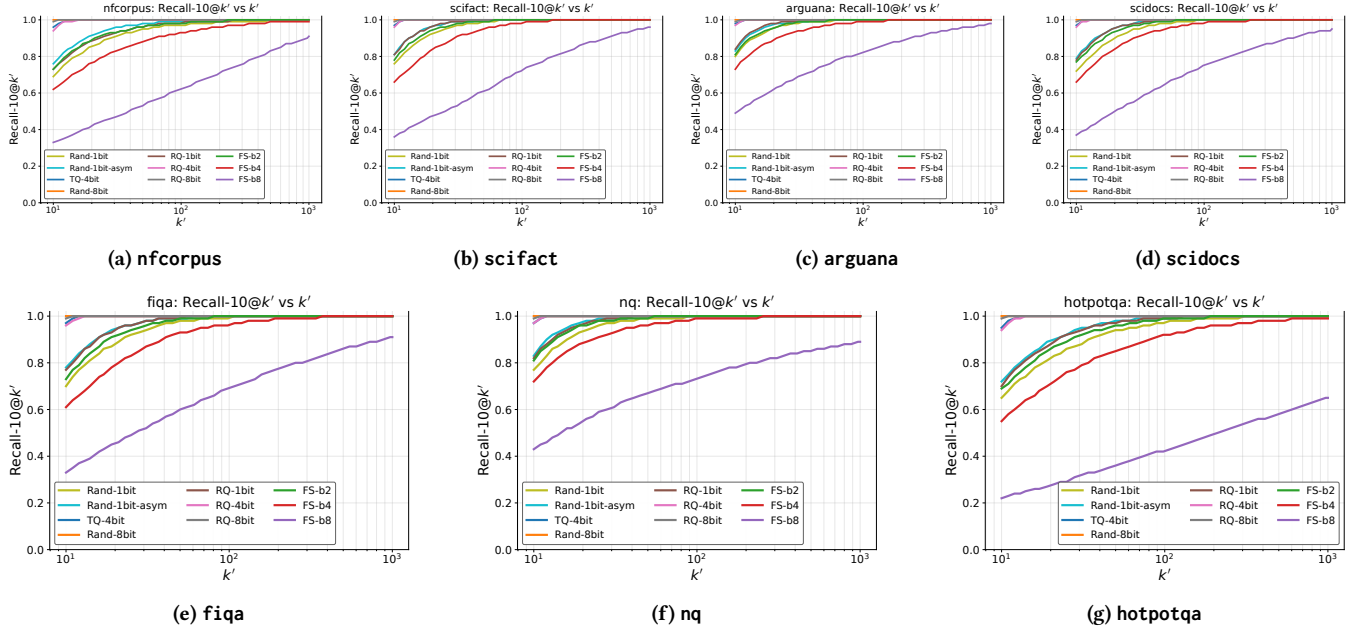


Figure 2: Multi-vector over-retrieval quality across datasets. Each panel shows recall as a function of the candidate budget k' .

0.5 by default, since it is slightly faster in our implementation and gives comparable or better recall across the operating points we consider. The cost of query compression itself is negligible compared to the overall search time; its main effect is to reduce the cost of the subsequent multi-vector scoring kernels. Furthermore, most of the loss in quality here can be recovered with reranking, allowing us to compress more aggressively.

B.4 Extended Latency Experiments

We present the latency vs recall plots on all of the datasets here; see Figures 4,5,6,7,8,9,10,11, and 12. See Tables 2 and 3 for a summary of the speedups obtained.

B.5 Extended Throughput Experiments

We present the batch throughput vs recall plots on all of the datasets here; see Figures 13,14,15,16,17,18,19,20, and 21. See Tables 2 and 3 for a summary of the speedups obtained.

B.6 Multi-thread Latency Experiments

We also present the multi-threaded latency vs recall plots, i.e. latency when single-queries run on all cores. This measures the amount of intra-query parallelization in the algorithm. See Figures 22,23,24,25, and 26.

REFERENCES

- [1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2016. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. In *Proceedings of the VLDB Endowment*, Vol. 9. New Delhi, India, 12. <https://inria.hal.science/hal-01239055>
- [2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2017. Accelerated Nearest Neighbor Search with Quick ADC. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval* (Bucharest, Romania)

- (ICMR '17). Association for Computing Machinery, New York, NY, USA, 159–166. <https://doi.org/10.1145/3078971.3078992>
- [3] Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. 2024. MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. In *Neural Information Processing Systems (NeurIPS)*.
- [4] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3, Article 167 (2024).
- [5] Magdalen Dobson Manohar, Zheqi Shen, Guy Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Edinburgh, United Kingdom) (PPoPP '24). Association for Computing Machinery, New York, NY, USA, 270–285. <https://doi.org/10.1145/3627535.3638475>
- [6] Joe H. Ward. 1963. Hierarchical Grouping to Optimize an Objective Function. *J. Amer. Statist. Assoc.* 58, 301 (1963), 236–244. <https://doi.org/10.1080/01621459.1963.10500845>

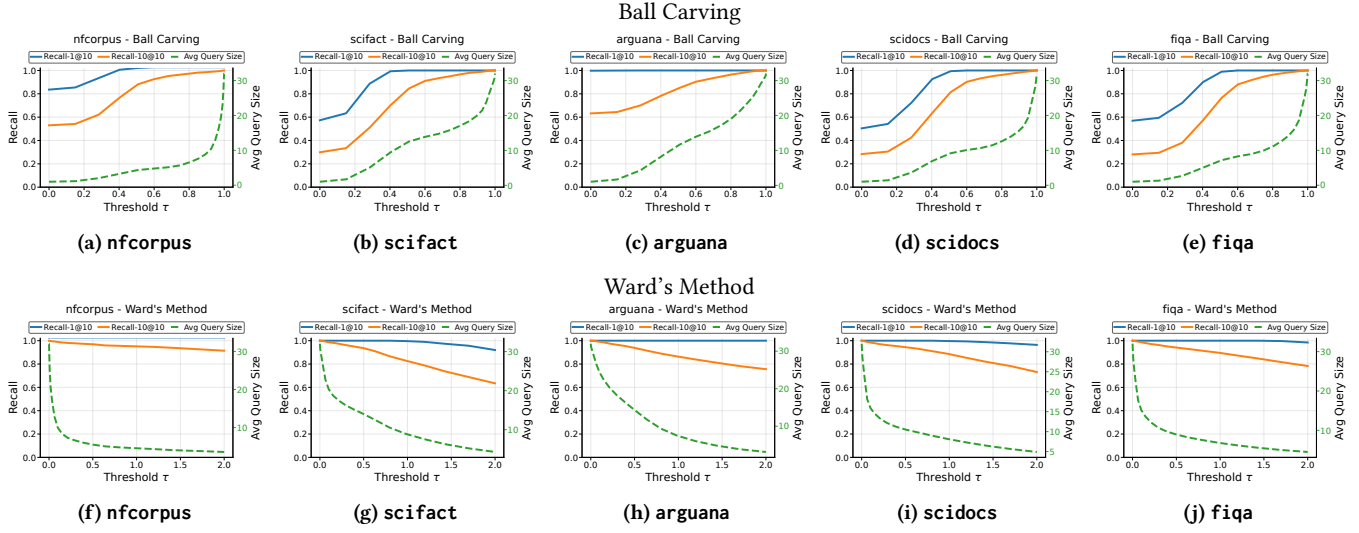


Figure 3: Query-compression quality for Ball Carving (top row) and Ward's method (bottom row). Each panel plots retrieval quality as the compression threshold varies.

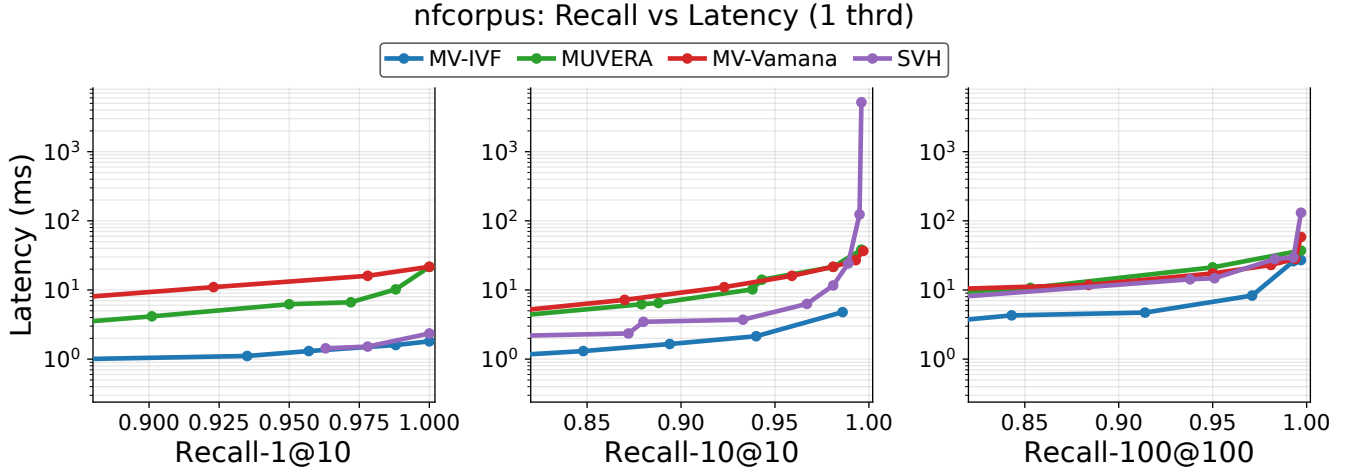


Figure 4: Latency: nfcampus.

Regime	Baseline	90% Recall-10@10	95% Recall-10@10	99% Recall-10@10
Latency	MUVERA	1.33× / 4.76× (9)	1.91× / 4.51× (6)	2.09× / 2.92× (3)
	SVH	1.55× / 8.03× (8)	2.89× / 8.86× (6)	12.46× / 31.09× (4)
	MV-Vamana	1.22× / 5.13× (9)	1.88× / 3.35× (6)	3.30× / 4.98× (4)
Throughput	MUVERA	2.19× / 8.40× (9)	1.92× / 6.97× (7)	2.10× / 6.28× (3)
	SVH	4.01× / 173.53× (8)	4.71× / 42.85× (7)	67.49× / 1482.60× (4)
	MV-Vamana	2.07× / 4.99× (9)	2.20× / 4.77× (8)	5.05× / 11.65× (4)

Table 2: MVIVF speedups. Each cell reports geometric-mean speedup / maximum speedup over the baseline, with the number of datasets compared in parentheses. The latency/throughput is computed by taking the best value with Recall-10@10 at least the fixed value.

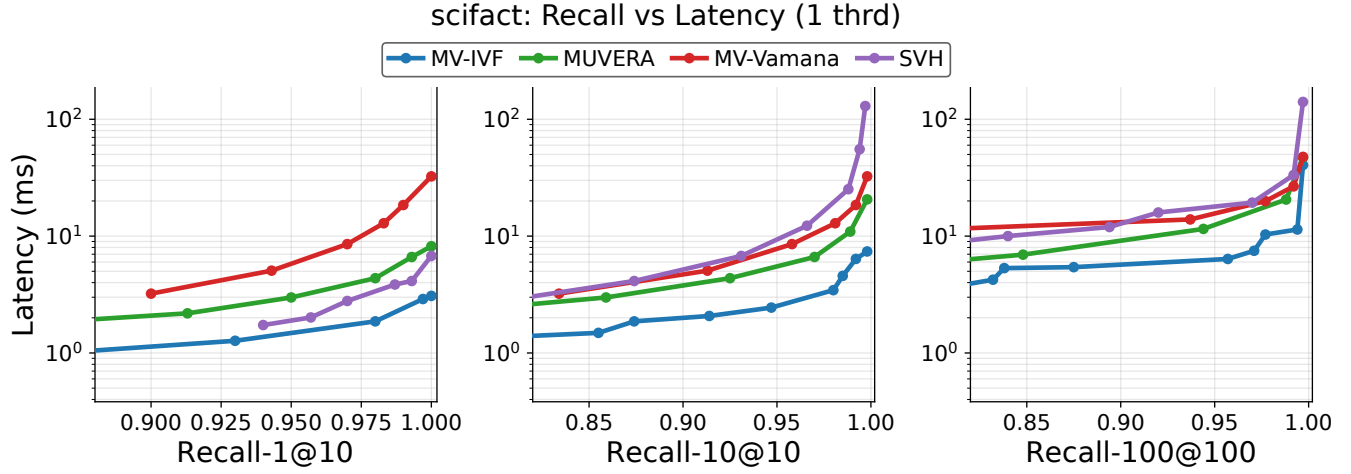


Figure 5: Latency: scifact.

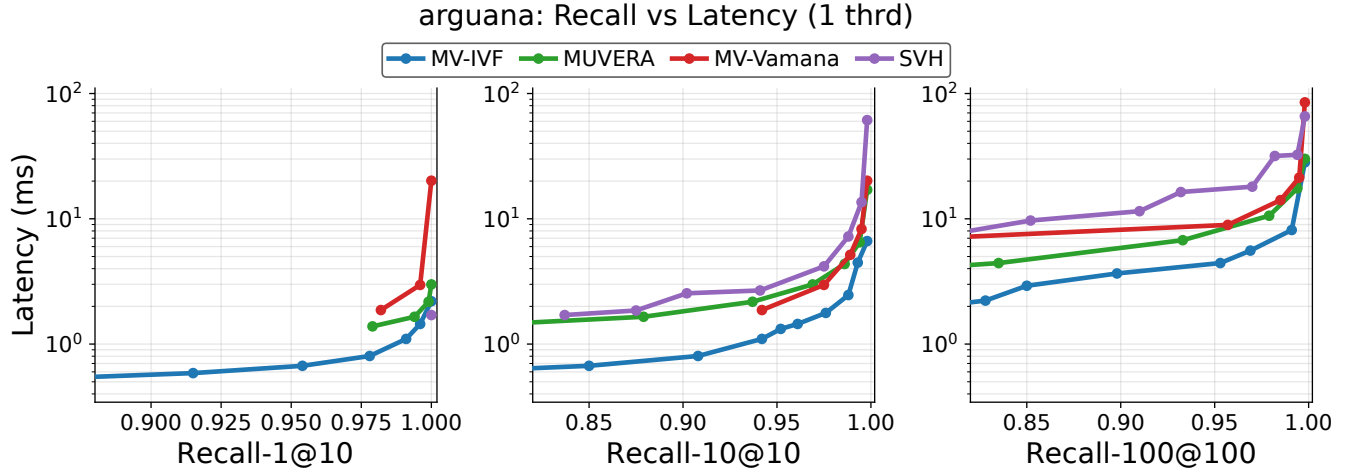


Figure 6: Latency: arguana.

Regime	Baseline	90% Recall-100@100	95% Recall-100@100	99% Recall-100@100
Latency	MUVERA	2.25× / 4.50× (6)	3.08× / 3.96× (5)	2.45× / 4.13× (4)
	SVH	3.03× / 4.06× (5)	3.54× / 5.06× (5)	2.63× / 3.99× (4)
	MV-Vamana	1.91× / 3.68× (6)	2.45× / 3.12× (5)	1.98× / 2.62× (4)
Throughput	MUVERA	3.43× / 26.55× (8)	4.53× / 16.00× (5)	3.22× / 5.93× (4)
	SVH	3.12× / 32.24× (8)	3.04× / 12.16× (6)	5.95× / 80.08× (5)
	MV-Vamana	1.96× / 5.34× (9)	2.83× / 4.86× (9)	2.65× / 4.03× (5)

Table 3: MVIVF speedups. Each cell reports geometric-mean speedup / maximum speedup over the baseline, with the number of datasets compared in parentheses. The latency/throughput is computed by taking the best value with Recall-100@100 at least the fixed value.

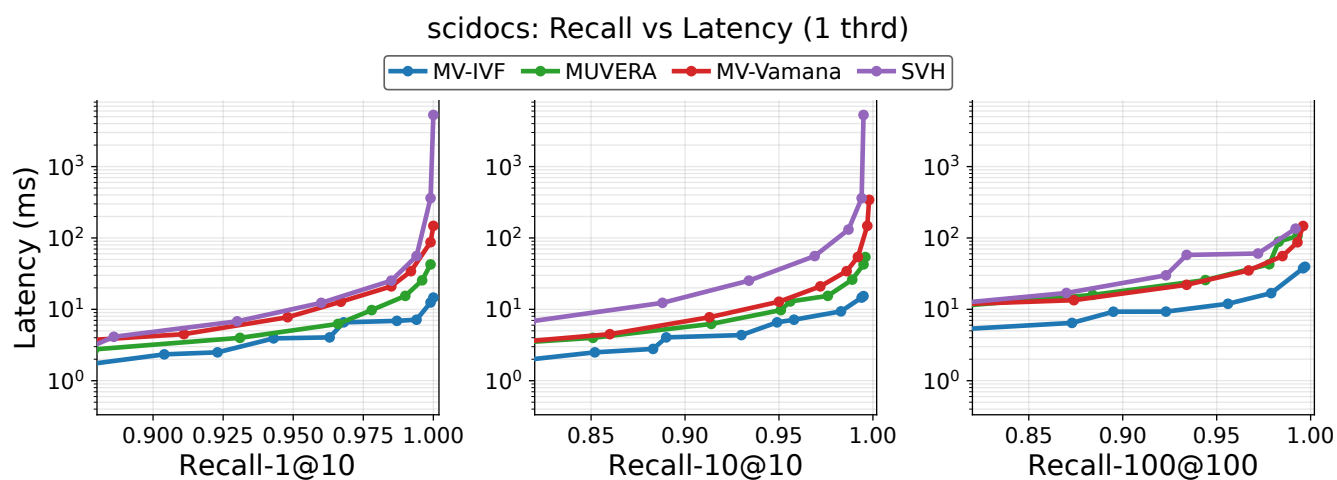


Figure 7: Latency: scidocs.

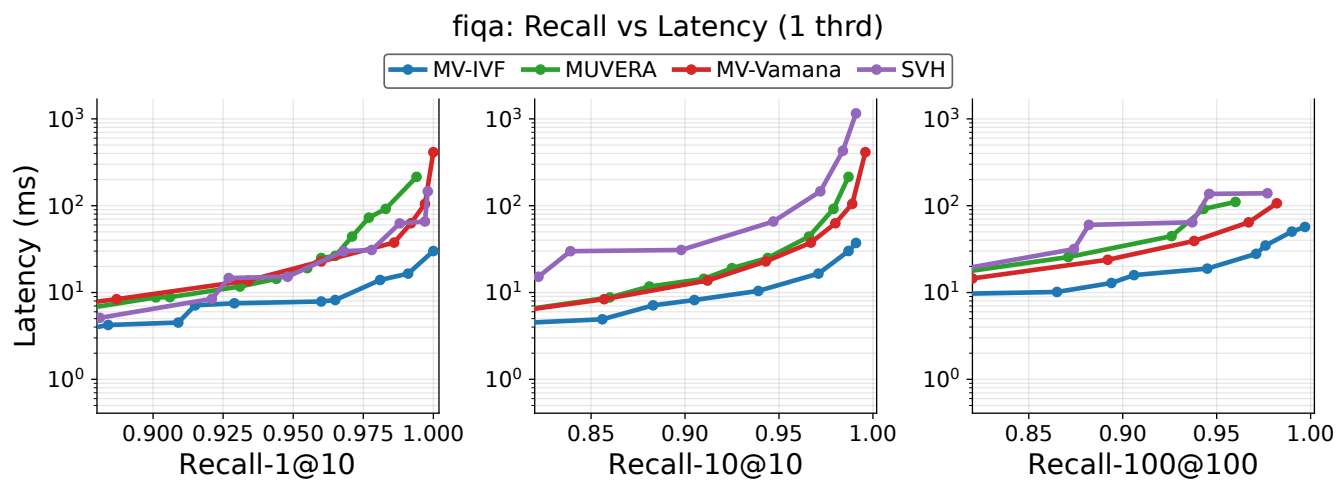


Figure 8: Latency: fiqa.

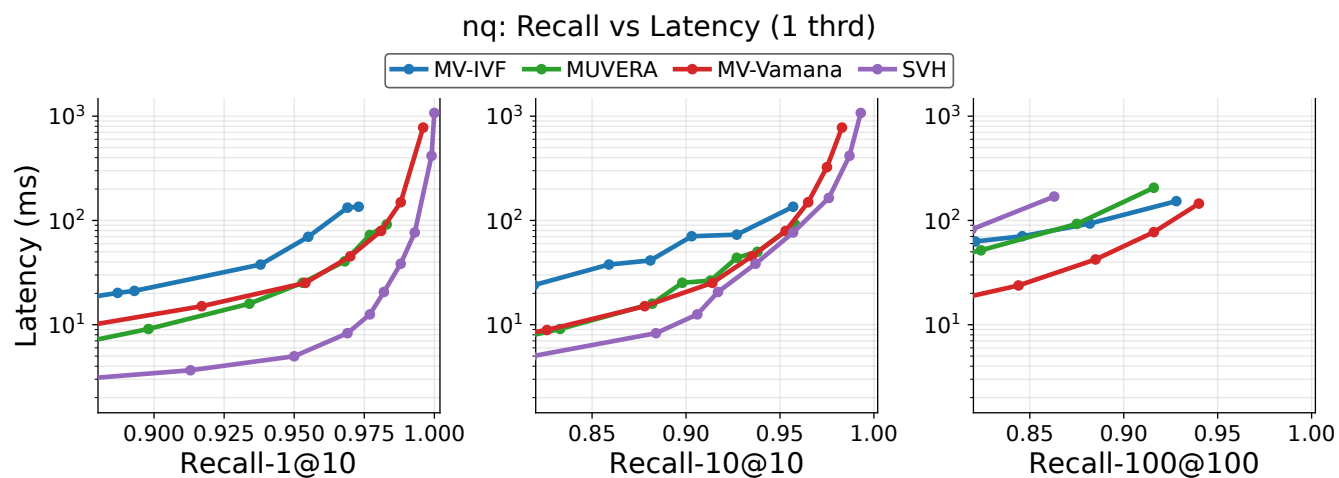


Figure 9: Latency: nq.

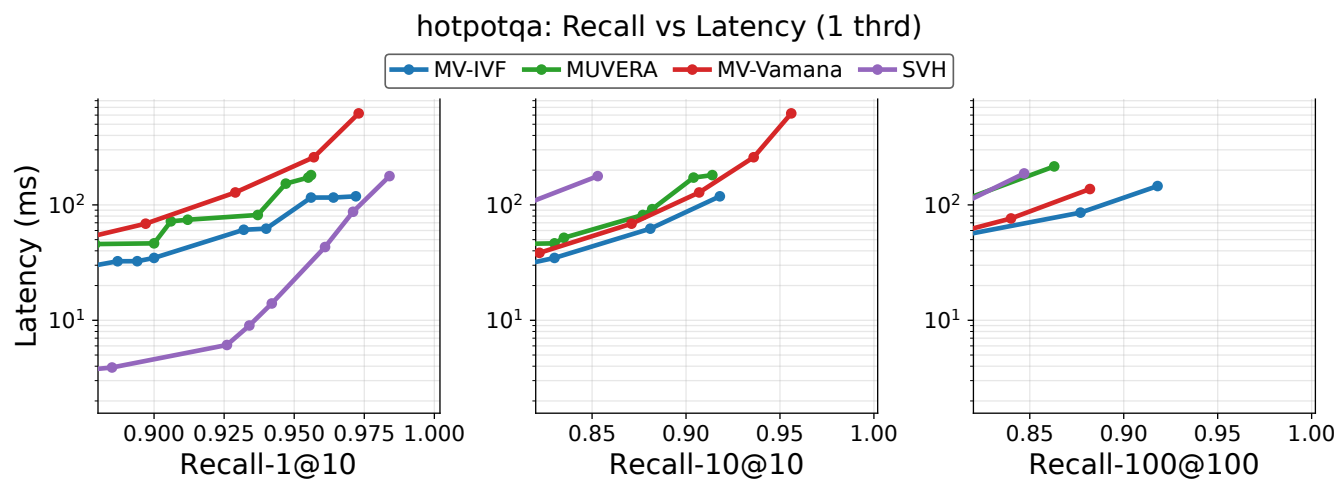


Figure 10: Latency: hotpotqa.

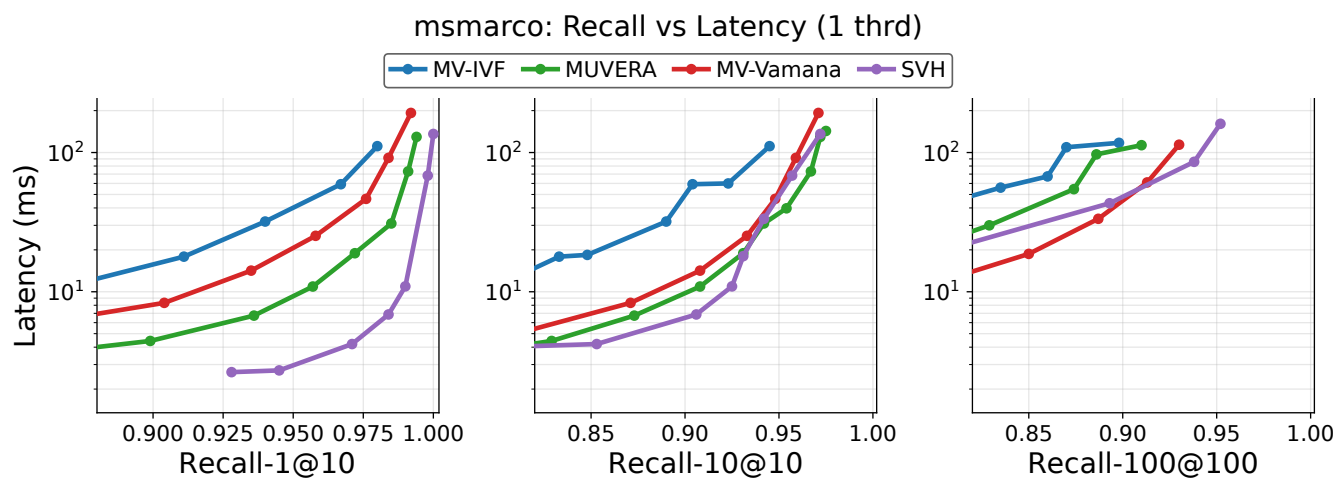


Figure 11: Latency: msmarco.

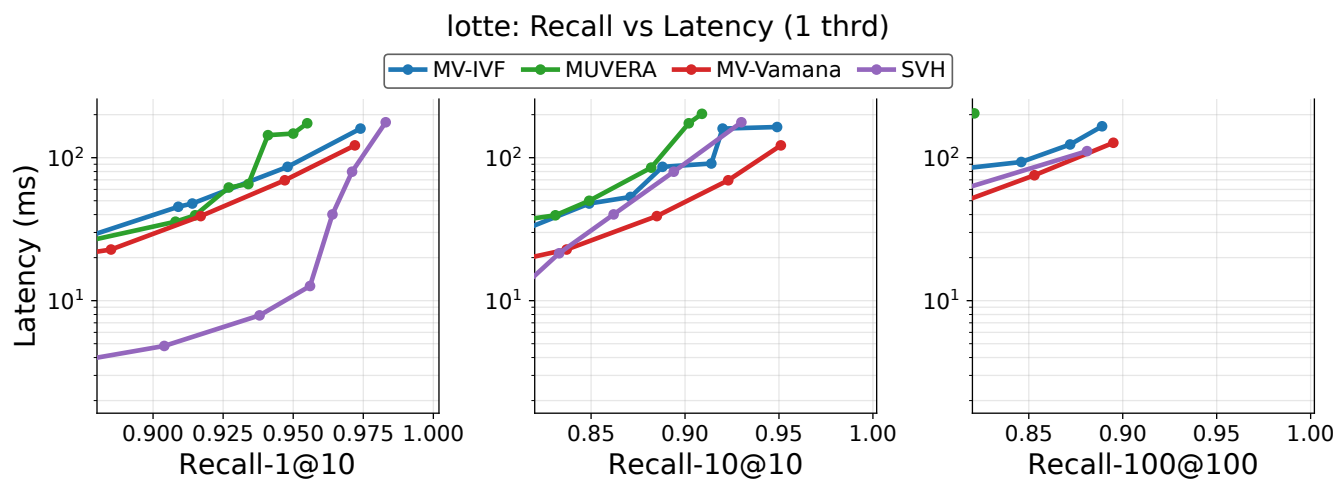


Figure 12: Latency: lotte.

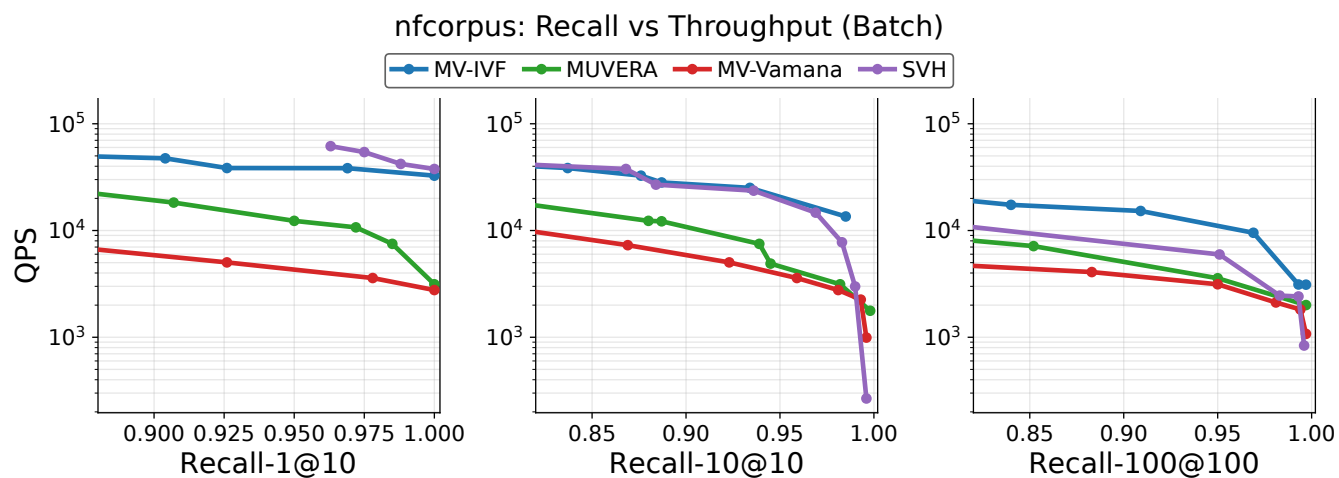


Figure 13: Throughput: nfcorpus.

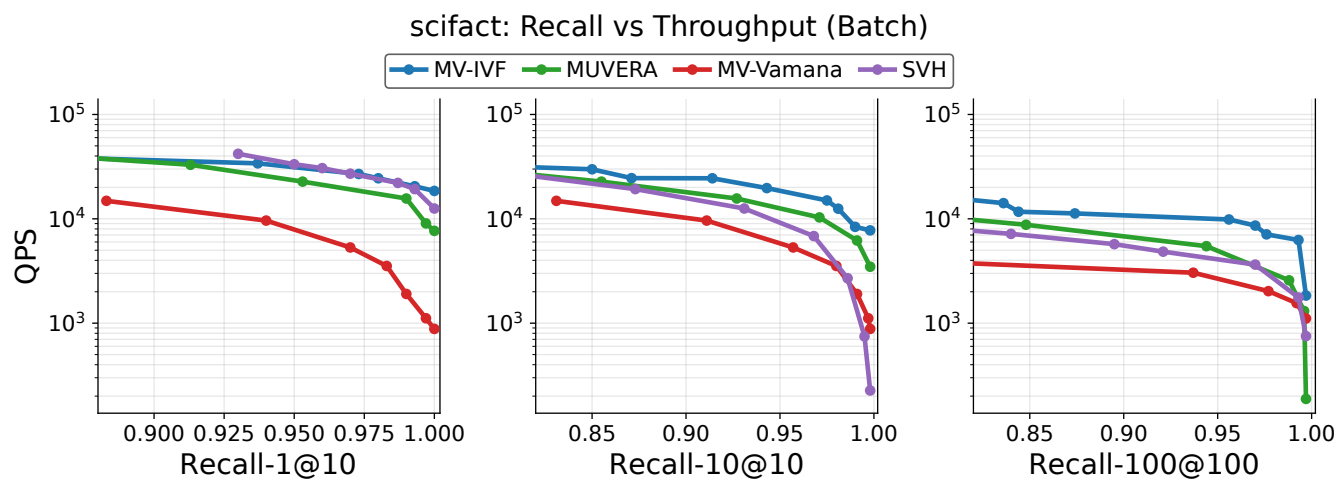


Figure 14: Throughput: scifact.

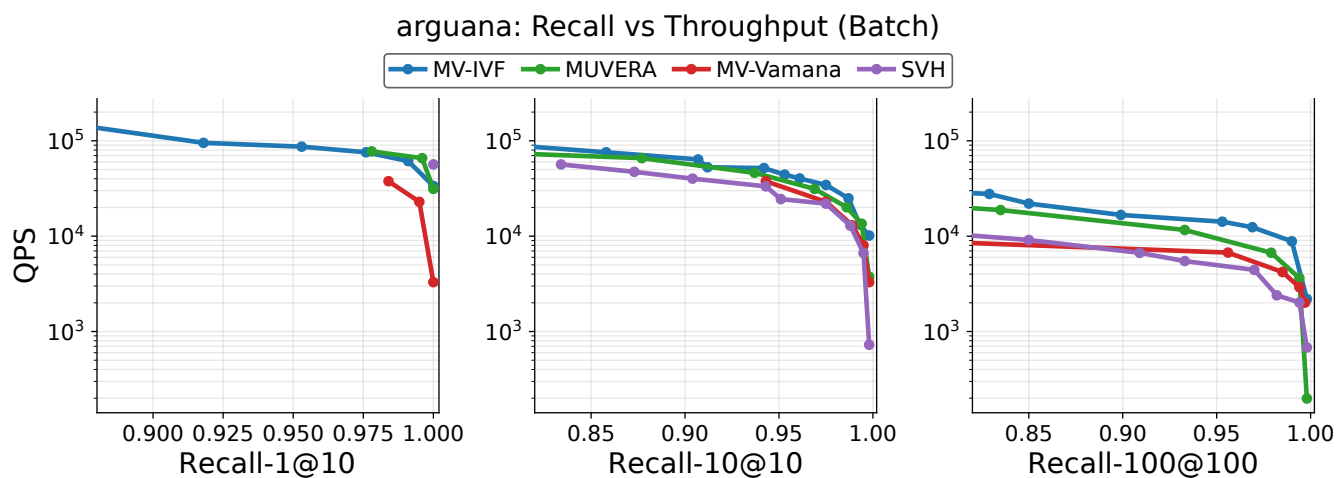


Figure 15: Throughput: arguana.

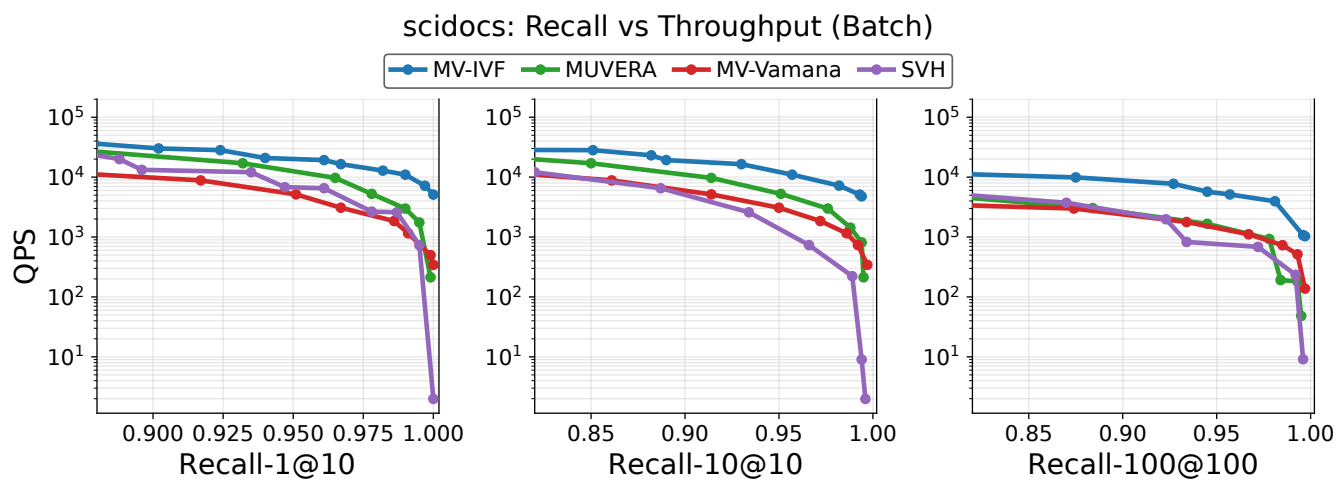


Figure 16: Throughput: scidocs.

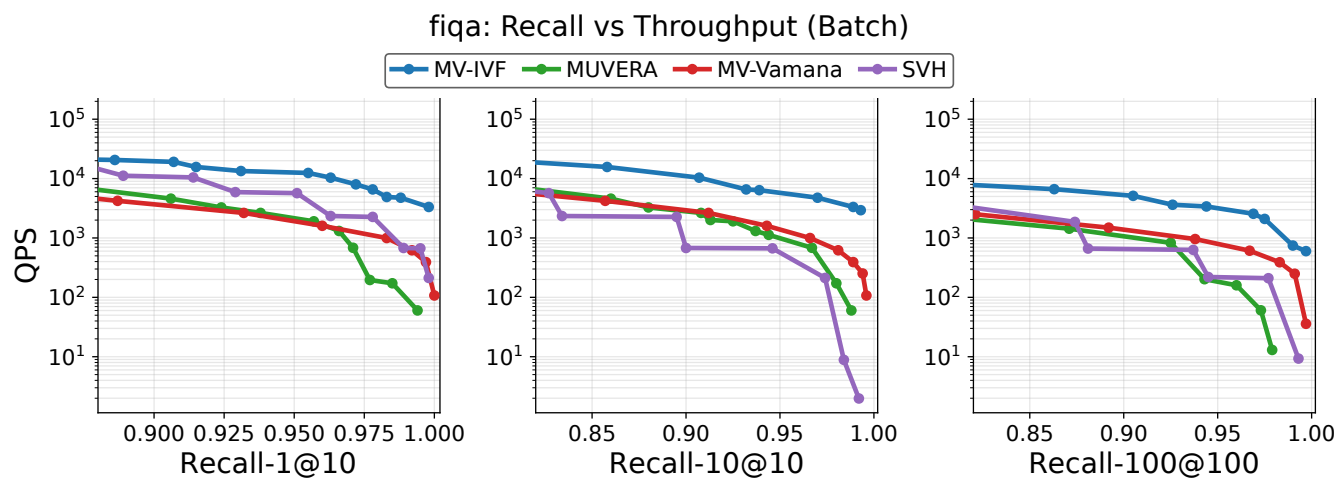


Figure 17: Throughput: fiqa.

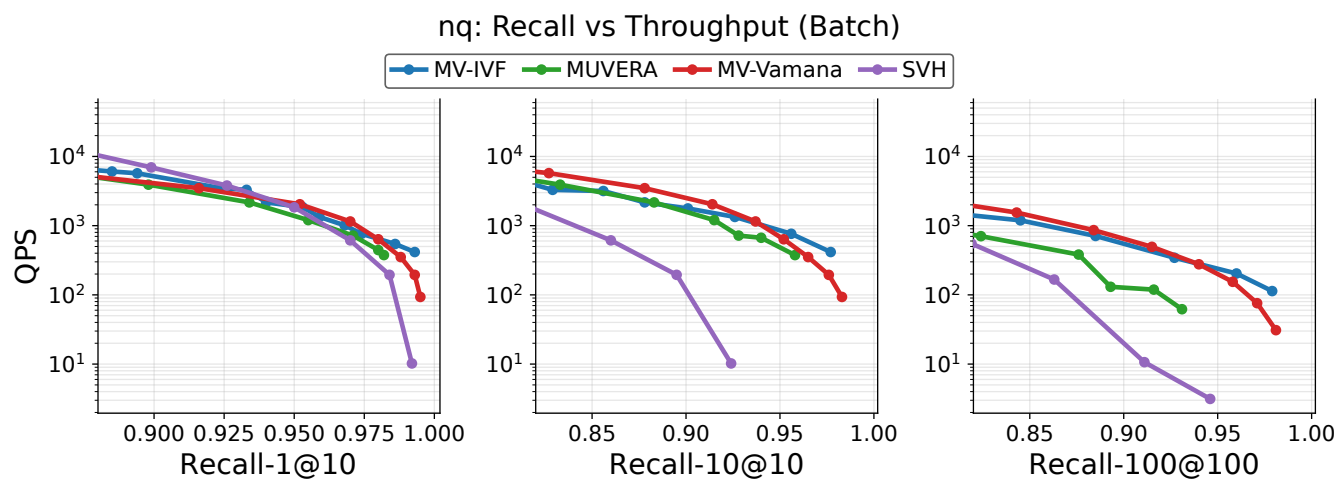


Figure 18: Throughput: nq.

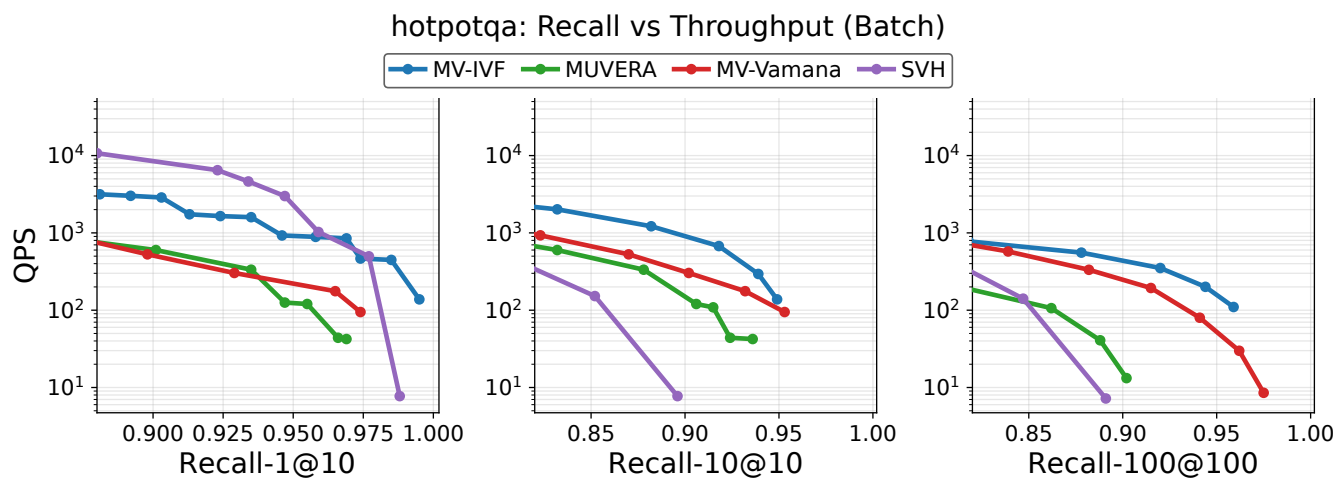


Figure 19: Throughput: hotpotqa.

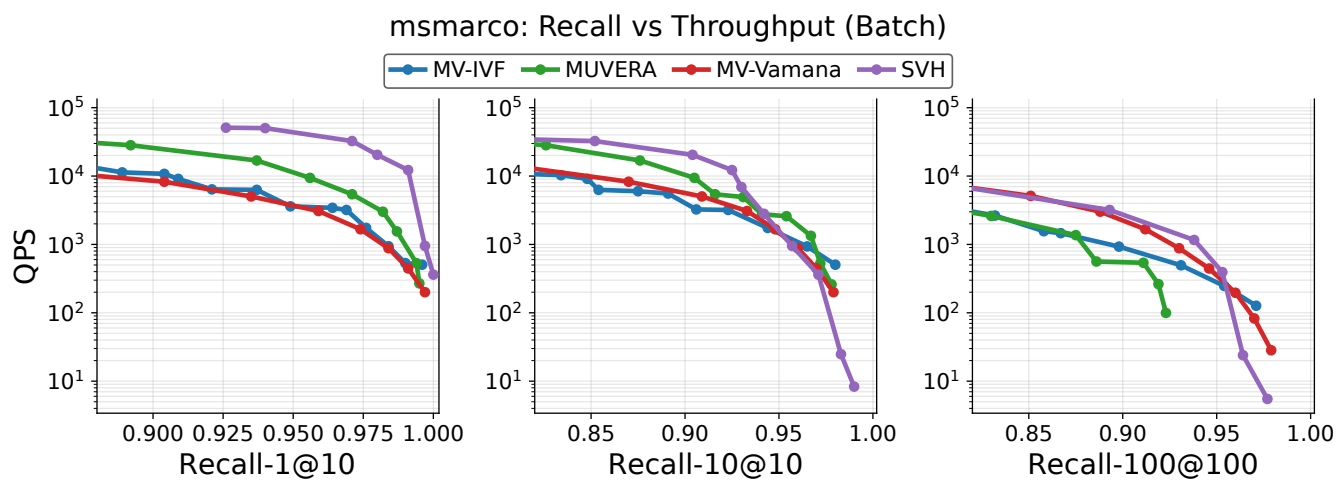


Figure 20: Throughput: msmarco.

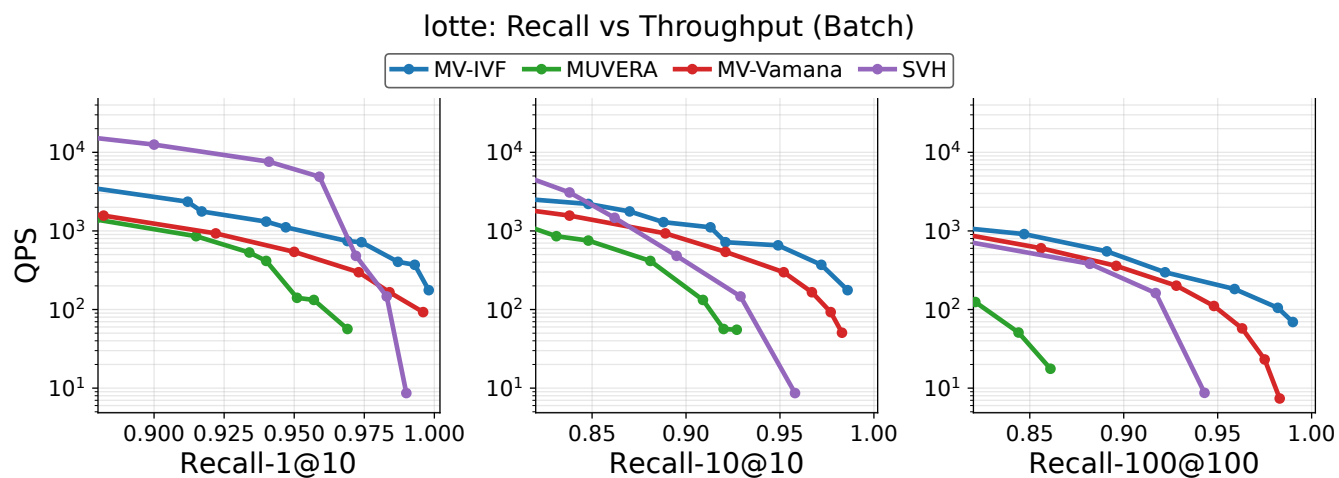


Figure 21: Throughput: lotte.

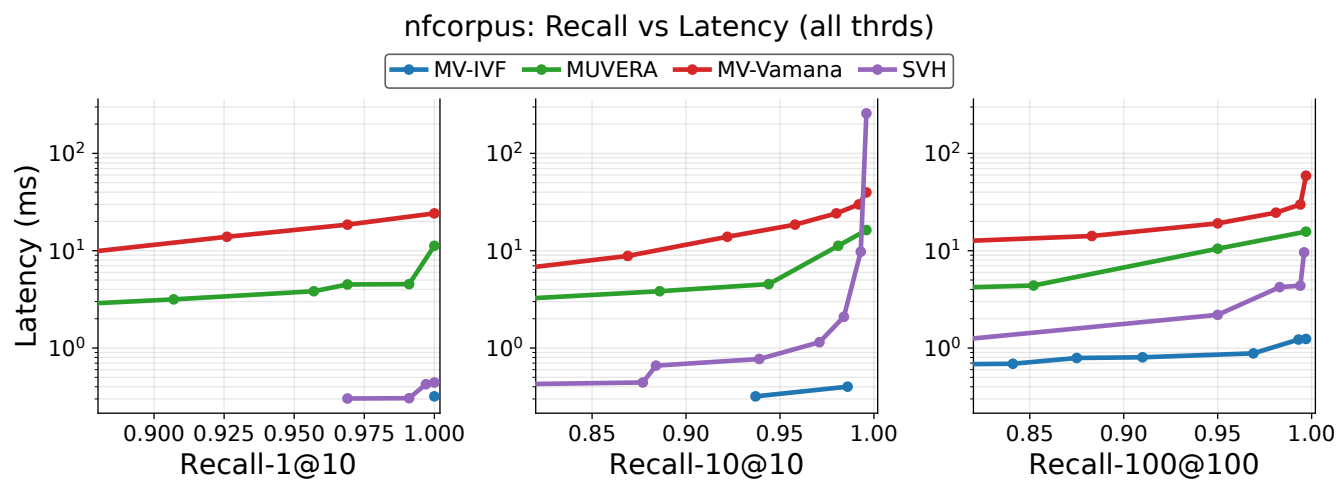


Figure 22: Multi-thread latency: nfcopus.

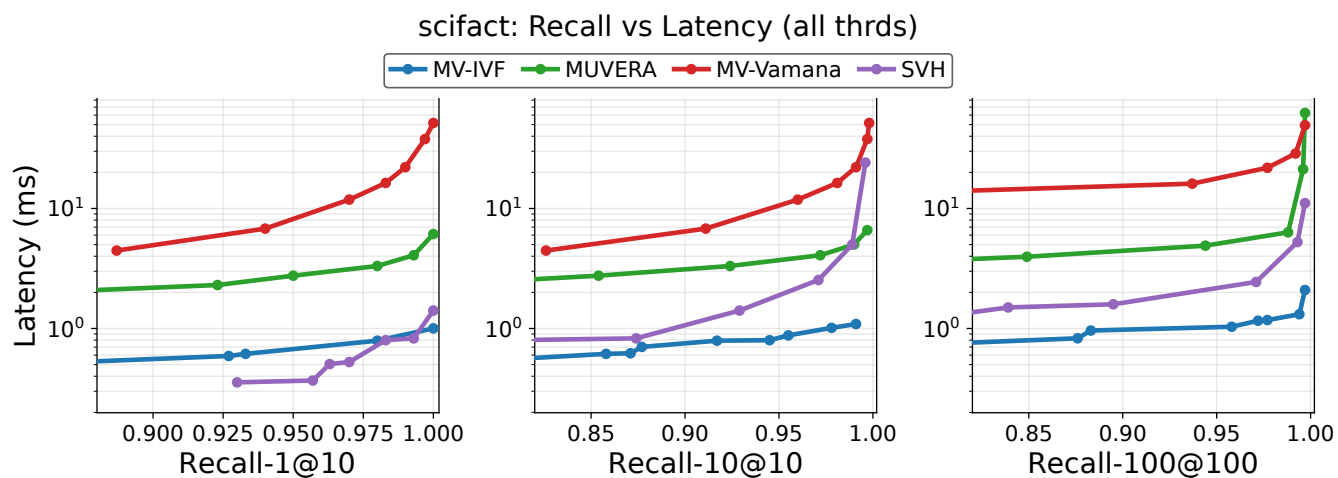


Figure 23: Multi-thread latency: scifact.

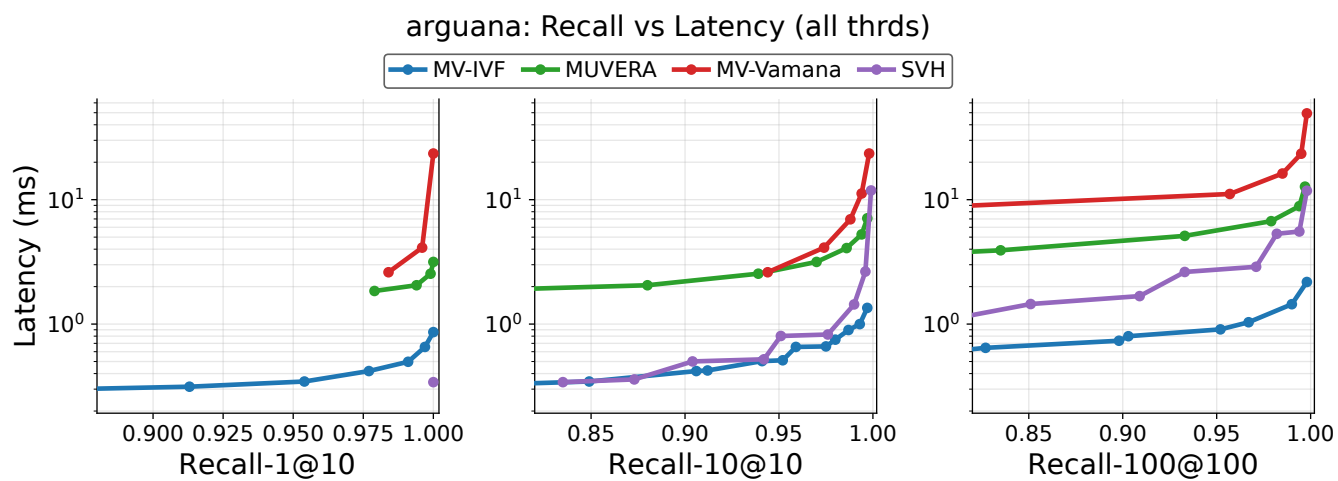


Figure 24: Multi-thread latency: arguana.

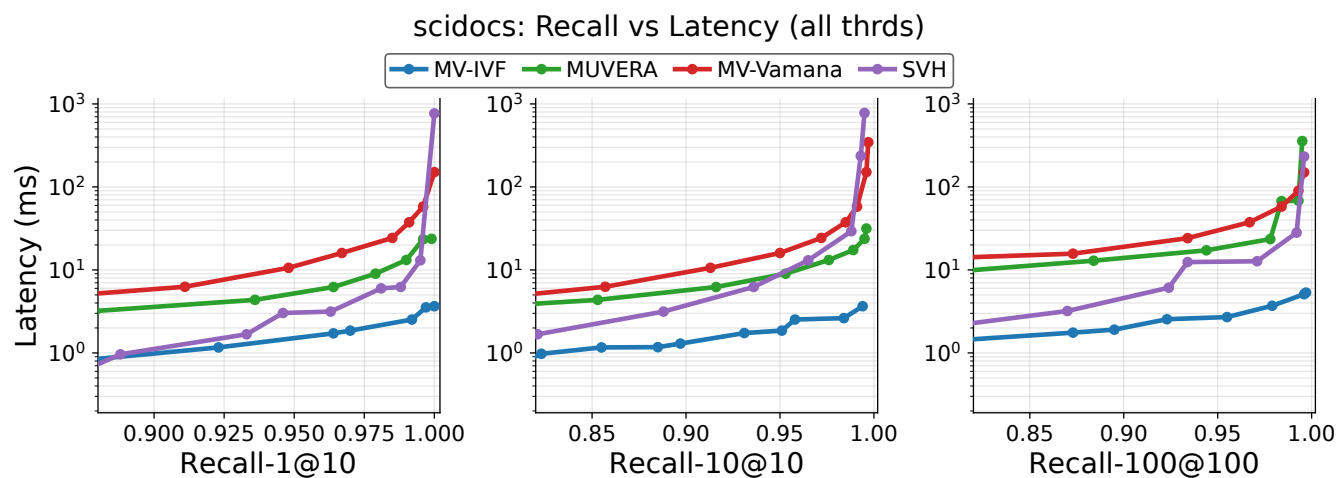


Figure 25: Multi-thread latency: scidocs.

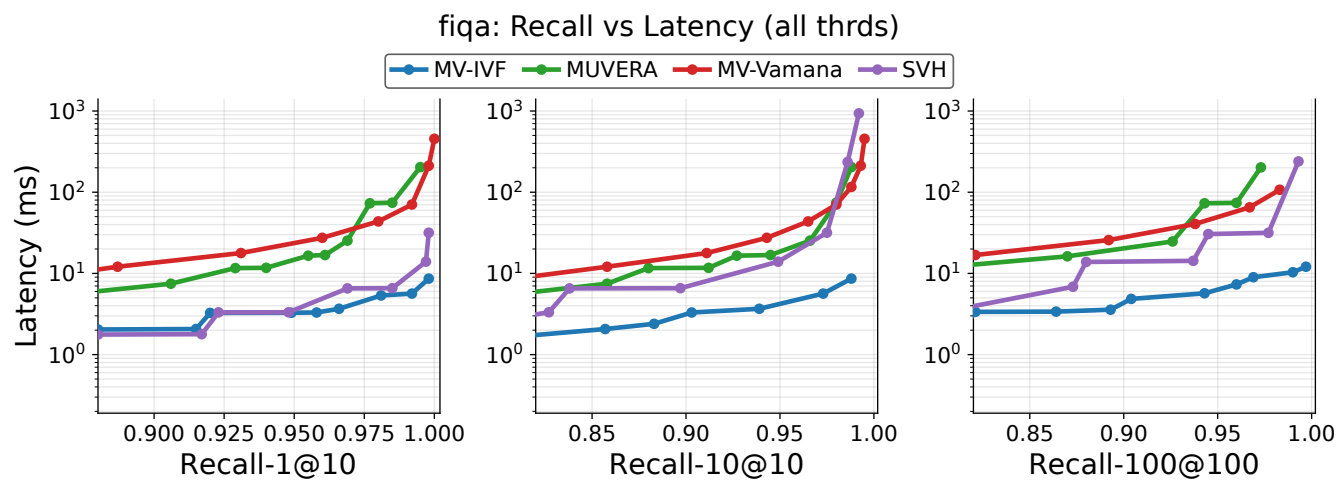


Figure 26: Multi-thread latency: fiqa.